



PDF Download
3392717.3392743.pdf
15 March 2026
Total Citations: 16
Total Downloads: 424

Latest updates: <https://dl.acm.org/doi/10.1145/3392717.3392743>

RESEARCH-ARTICLE

Efficient parallel algorithms for betweenness- and closeness-centrality in dynamic graphs

KSHITIJ SHUKLA, International Institute of Information Technology, Hyderabad, Hyderabad, TG, India

SAI CHARAN REGUNTA, International Institute of Information Technology, Hyderabad, Hyderabad, TG, India

SAI HARSH TONDOMKER, International Institute of Information Technology, Hyderabad, Hyderabad, TG, India

KISHORE KOTHAPALLI, International Institute of Information Technology, Hyderabad, Hyderabad, TG, India

Open Access Support provided by:

International Institute of Information Technology, Hyderabad

Published: 29 June 2020

Citation in BibTeX format

ICS '20: 2020 International Conference on
Supercomputing
June 29 - July 2, 2020
Spain, Barcelona

Conference Sponsors:
SIGARCH

Efficient Parallel Algorithms for Betweenness- and Closeness-Centrality in Dynamic Graphs

Kshitij Shukla

kshitij.shukla@research.iiit.ac.in

International Institute of Information Technology,
Hyderabad

Hyderabad, India 500 032.

Sai Harsh Tondomker

sai.harsh@research.iiit.ac.in

International Institute of Information Technology,
Hyderabad

Hyderabad, India 500 032.

Sai Charan Regunta

saicharan.regunta@research.iiit.ac.in

International Institute of Information Technology,
Hyderabad

Hyderabad, India 500 032.

Kishore Kothapalli

kkishore@iiit.ac.in

International Institute of Information Technology,
Hyderabad

Hyderabad, India 500 032.

ABSTRACT

Finding the centrality measures of nodes in a graph is a problem of fundamental importance due to various applications from social networks, biological networks, and transportation networks. Given the large size of such graphs, it is natural to use parallelism as a recourse. There have been several studies that show how to compute the various centrality measures of nodes in a graph on parallel architectures, including multi-core systems and GPUs. However, as these graphs evolve and change, it is pertinent to study how to *update* the centrality measures on changes to the underlying graph.

In this paper, we show novel parallel algorithms for updating the betweenness- and closeness-centrality values of nodes in a dynamic graph. Our algorithms process a *batch* of updates in parallel by extending the approach of handling a single update for betweenness- and closeness-centrality by Jamour et al. [16] and Sariyüce et al. [27], respectively. Besides, our algorithms incorporate mechanisms to exploit the structural properties of graphs for enhanced performance.

We implement our algorithms on two parallel architectures: an Intel 24-core CPU and an Nvidia Tesla V100 GPU. To the best of our knowledge, we are the first to show GPU algorithms for the above two problems. We conduct detailed experiments to study

the impact of various parameters associated with our algorithms and their implementation. Our results on a collection of real-world graphs indicate that our algorithms achieve a significant speedup over corresponding state-of-the-art algorithms.

CCS CONCEPTS

• **Theory of computation** → **Parallel algorithms**.

KEYWORDS

Dynamic Graphs, Centrality Measures, Batch update, GPU, CPU, Parallel Algorithms

ACM Reference Format:

Kshitij Shukla, Sai Charan Regunta, Sai Harsh Tondomker, and Kishore Kothapalli. 2020. Efficient Parallel Algorithms for Betweenness- and Closeness-Centrality in Dynamic Graphs. In *2020 International Conference on Supercomputing (ICS '20)*, June 29–July 2, 2020, Barcelona, Spain. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3392717.3392743>

1 INTRODUCTION

Graphs are a useful abstraction to model a variety of phenomenon from domains such as social networks [7], biological networks [4], transportation networks, and the like. A significant problem with graphs from these domains is computing the various centrality metrics. Centrality metrics assign a numerical value to the nodes/edges of a graph. Popular centrality measures include the betweenness-centrality [4], closeness-centrality [7], Katz-centrality [2], pagerank-centrality [2], and the like.

In this paper, we are interested in two centrality measures: betweenness-centrality and closeness-centrality. Let $G = (V, E)$ be an undirected and unweighted graph. For $s, t \in V$ let $\delta_{st}(v)$ number of shortest paths from s to t that pass through v . Let δ_{st} denote the

This work is funded in part by the Department of Science and Technology, Government of India, via Grant number EMR/2016/007668/EEC..

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICS '20, June 29–July 2, 2020, Barcelona, Spain

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7983-0/20/06...\$15.00

<https://doi.org/10.1145/3392717.3392743>

Author	Batch Update	Fully Dynamic	Parallel	Exact
Betweenness Centrality				
Green et al. [13]	✗	✗	✗	✓
Jamour et al. [16]	✗	✓	✓	✓
Pontecorvi et al. [23]	✗	✓	✗	✓
Bergamini et al. [3]	✓	✗	✗	✗
Our	✓	✓	✓	✓
Closeness Centrality				
Kas et al. [17]	✗	✓	✗	✓
Sarıyüce et al. [27]	✗	✗	✓	✓
Chia-Chen et al. [32]	✗	✓	✗	✓
Our	✓	✓	✓	✓

Table 1: A brief comparison of various closely related algorithms for dynamic centrality computation on graphs.

number of shortest paths from s to t . The betweenness-centrality of a node v in a graph, denoted $bc(v)$, is computed as $bc(v) = \sum_{s,t} \frac{\delta_{st}(v)}{\delta_{st}}$. The closeness-centrality of a node v in a graph G , denoted $cc(v)$, is the ratio of $|V| - 1$ to the sum of the distances from v to every other node in G . Such metrics are used, for instance, to find the most important/influential nodes/edges in social networks, or to find the key nodes/edges from an infrastructure view point in a transportation network, and the like.

These applications created immense research interest in algorithms for computing betweenness- and closeness-centrality metrics in many models of computing, such as sequential, distributed, embedded, and parallel computing. The approach of Brandes [4] has been used in several sequential and parallel algorithms for computing the betweenness-centrality values of nodes in a graph (cf. [20, 22]). There have been studies also on how to approximate $bc(v)$ and $cc(v)$ [1, 7] using a variety of techniques.

However, graphs that represent phenomenon such as social networks, biological networks, and transportation networks change over time via the addition or deletion of a small number of edges [18]. In such a setting, it is vital to update the centrality values to keep the metrics as current as possible. However, computing centrality measures is an expensive operation. For instance, the algorithm of Brandes [4] requires work in $O(|V| \cdot |E|)$ for computing the betweenness-centrality values of nodes in an unweighted graph. Therefore, it is pertinent to design efficient algorithms for updating the centrality measures on evolving/dynamic graphs instead of recomputing the metric across the entire graph.

There are a few reported works for computing the betweenness- and closeness-centrality values of nodes in a dynamic graph. Following the nomenclature of dynamic graph algorithms, incremental (*decremental*) algorithms handle the case where the current graph changes via either an insertion (*resp.* deletion) of a node/edge. Fully dynamic algorithms handle both addition and deletion of nodes/edges. Table 1 shows various features of dynamic algorithms for updating centrality measures.

Some of the recent papers in this direction in the parallel setting [16, 27] for arriving at the centrality values of nodes in a dynamic graph under a *single edge* update start by identifying the *affected* nodes, perform a traversal from each such affected node, and finally update the centrality values of nodes. However, one can think of several use-cases where a collection of updates to the current graph can be accumulated in a unit time and process the collection for an update to the graph analytic. For instance, Facebook reports 15 million new users every month¹ translating to a daily average of 500,000 updates. Such use-cases indicate that there is scope for designing appropriate algorithms for processing a batch of updates. Similarly, the huge volume of computation involved in the problems of interest calls for other light-weight optimizations that offer practical benefits. In light of the above observations, we note that recent parallel approaches to the problem have some limitations. In the following, we show how our contributions in this paper address these limitations by introducing algorithmic and other performance-enhancing improvements.

1.1 Our Contributions

In this paper, we solve the problem of dynamic betweenness- and closeness-centrality measures. To the best of our knowledge, we are the *first* ones to show fully dynamic algorithms for betweenness- and closeness-centrality measures on a GPU. Below is a summary of our novel contributions.

- We design parallel batch update algorithms for dynamic betweenness-centrality of graphs and thereby solve an open problem mentioned by Jamour et al. [16].
- We extend our algorithms to update the closeness-centrality values of nodes in a graph under a batch update, whereas the work of Sarıyüce et al. [27] handles the case of a single edge update (cf. Section 4).
- Our algorithms also include mechanisms to carefully exploit the structural properties of the underlying graph. These optimizations can help alleviate the huge volume of computation in the problems of interest (cf. Section 2).
- Further, for the closeness-centrality problem in dynamic graphs, we express some of the computation via top-down and bottom-up traversals of the block-cut tree of the graph. This technique can be of independent interest in solving related problems.

We implement our algorithms on two parallel architectures: an Intel multi-core CPU and an NVidia GPU. We conduct detailed experiments studying the impact of various parameters associated with the algorithm and its implementation. For sake of comparison, we also implement Jamour et al. [16] and Sarıyüce et al. [27] algorithms on GPU. Table 2 shows a summary of our results.

1.2 Related Work

Sequential algorithms for dynamic graphs have a rich history including the important works of Frederickson [12], Roditty and Zwick

¹Top 20 Facebook Statistics, Number 8, <https://zephoria.com/top-15-valuable-facebook-statistics/>

Betweenness-centrality			Closeness-centrality		
Batch size	CPU	GPU	Batch size	CPU	GPU
25	15.76	14.65	100	17.98	21.54
50	32.38	31.48	250	29.78	33.33
100	65.31	64.95	500	40.94	49.49

Table 2: Average speedup of our algorithms over Jamour et al. [16] and over Sariyüce et al. [27] for betweenness- centrality and closeness-centrality respectively. Detailed results are shown in Sections 3 and 4.

[25], and the sparsification framework of Eppstein et al. [10]. Ramalingam [24] introduces the notion of incremental complexity that measures the time required to process an update in terms of the impact the update has on the analytic. This is called the incremental complexity of the problem. Green et al. [13] are the first ones to propose an incremental algorithm for updating the betweenness-centrality values of nodes. Green et al. use the notion of affected nodes that is later used by other works, including [16, 27, 30], and in this paper too. Yen et al. [32] use the CENDY framework to update the closeness-centrality, and Kas et al. [17] use dynamic all-pairs-shortest-paths to update the closeness-centrality values. Sequential algorithms for incremental betweenness-centrality are studied in [23] whereas, Bergamini et al. [3] study approximate betweenness-centrality under a batch update.

Concerning parallel algorithms, Liang et al. adopt the sparsification framework to design PRAM-style algorithms that require logarithmic time and $O(n)$ work for maintaining the 2- and 3-components under a single edge update. Mc Coll et al. [19] adopt a batch update model to maintain the connected components of a graph in parallel. Srinivasan et al. [29], and Ferragina [11] study graph connectivity under the addition and deletion of edges. They adopt the sparsification framework developed by Eppstein et al. [10]. Further, Srinivasan et al. [30] use the notion of affected nodes to update the shortest paths in a dynamic graph in parallel. Jamour et al. [16] and Sariyüce et al. [27] respectively show how to update the betweenness- and closeness-centrality values of nodes in a dynamic graph under a single edge update on multi-core CPUs. Ediger et al. [9] show how to maintain graph data structures under a batch of updates. Busato et al. [5] use a bit-vector based data structure for representing a dynamic graph and show how to maintain such a data structure under a batch of updates [5].

Parallel algorithms for graphs on a variety of many-core and multi-core architectures are of great interest over the decades. In recent years, one particular line of work that is popular in this context is to consider the structural properties of the input graph and study how these properties impact the algorithm design and implementation process. Hong et al. [15], Djidjev et al. [8], Slota et al. [28] and Pachorkar et al. [22], and Sariyüce et al. [26] in the context of computing centrality measures, show examples of this technique.

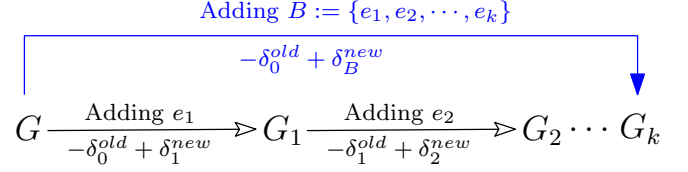


Figure 1: On adding a single edge, Jamour et al. [16] subtracts the old dependencies ($-\delta_0^{old}$) of the affected nodes in the graph G and adds dependencies in the new graph $G_1 := G + e_1$, denoted ($+\delta_1^{new}$). Adding another edge e_2 requires a similar transition from G_1 to $G_2 := G_1 + e_2$. Using a batch update, we propose to remove the old dependencies, δ_0^{old} and add the new dependencies, δ_B^{new} over the entire batch B .

1.3 Organization of the Paper

The rest of the paper is organized as follows. In Section 2, we provide a general description of the techniques we use. In Sections 3 and 4, we explain how these techniques apply to dynamic betweenness- and closeness-centrality computations. The paper ends with concluding remarks in Section 5.

2 OUR TECHNIQUES

In this section, we summarize the techniques that we use in the rest of the paper. We refer to G as the graph to which a batch B as a set of edges to be added (*deleted*), and the graph $G' := G \cup B$ (*resp.* $G \setminus B$) as the new graph. We discuss specific differences in how the techniques can be used in the case of betweenness-centrality and closeness-centrality, respectively, in Sections 3 and 4.

2.1 Batch Update

Existing approaches that update the betweenness- and closeness-centrality value of a dynamic graph in parallel are limited to the case of a single update (See also Table 1). For processing a single update, the most common technique used is to obtain a set of affected nodes quickly. As shown in [16, 27], when a new edge vw is added (or deleted) to a graph, then node u is said to be affected if $d(u, v) \neq d(u, w)$ for betweenness-centrality and if $|d(u, v) - d(u, w)| > 1$ for closeness-centrality where $d(a, b)$ is the distance between nodes a and b . By the above characterization, nodes v and w are also affected.

However, we note that processing a batch of edges offers several advantages compared to processing a single update. One can obtain work saving and scalability, as illustrated in the following. Consider a batch of edges B . When using the notion of affected nodes, consider a node v that is affected due to multiple edges $e_1, e_2, \dots$, in B . When processing each of these updates individually, a traversal has to be initiated from v in the graph $G + e_1, G + e_2$, and so on. However, as shown in Figure 1, a batch update algorithm can consider the set of nodes affected due to the entire batch and make a single update saving a significant amount of work.

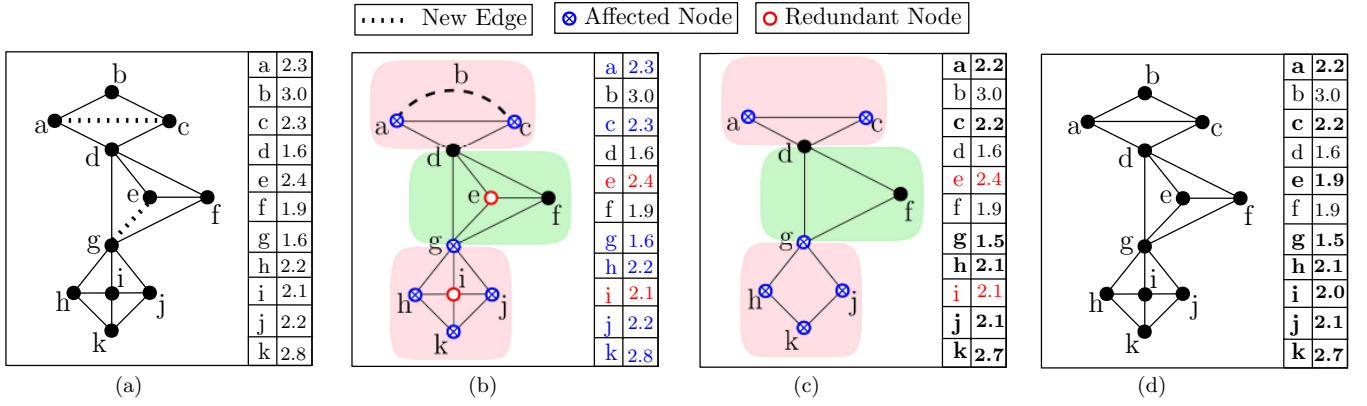


Figure 2: (a) A graph with the current reciprocal closeness-centrality values and newly added edges ac, eg , (b) Shows the BCC decomposition of G' and affected nodes along with chain $(a - b - c)$ and R_3, R_4 nodes e and i resp., (c) updates the reciprocal closeness-centrality values of the nodes within biconnected components in reduced graph, (d) updates the reciprocal closeness-centrality values across the components and also for the nodes ignored earlier.

2.2 Biconnected Component Decomposition

Biconnected component (BCC) decomposition is a technique that decomposes a graph into its biconnected components. The components, in turn, are connected via articulation points. BCC decomposition has been very popular recently in centrality computations [16, 22, 27] for its ability to confine the shortest paths between intra-BCC nodes to the same BCC. Using this technique, one can limit the scope of a breadth-first traversal (BFS) from a node v to the BCC(s) containing v . Doing so reduces the time spent in BFS traversals. Therefore, this decomposition offers significant computational advantages when updating centrality metrics in dynamic graphs, as we will demonstrate in later sections.

2.3 Chains and Redundant Chains

Chains are maximal paths in a graph. Hence, each chain has a set of nodes of degree two and two nodes, start and end, which have a degree not equal to two. We call the start and end nodes as *end-points* and nodes on a path as *chain nodes*. Chains in a graph offer great optimization opportunities in the context of path problems. Consider a chain C with a and b as *endpoints*. In this case, shortest paths passing through nodes in C also pass through the end-points a and/or b , unless the shortest paths have their origin and destination among the nodes of C . For example, the graph in Figure 2 has a chain $a - b - c$.

Further, we notice that graphs may also contain two or more chains $C_1, C_2, \dots$, that have common end-points, say a and b . In this case, shortest paths that go via a and also b will not pass through any chain that is not of the shortest length among $C_1, C_2, \dots$, unless the shortest paths in question start/end in such chain nodes. We call all such chains except those of the shortest length as *redundant chains*. Nodes on redundant chains can be processed differently in subsequent computations as detailed in Sections 3 and 4.

2.4 Redundant Nodes

We observe from our experiments that real-world graphs have a significant number of nodes with degree three and four. Setting aside some of these nodes from expensive processing can help in scaling the computations to large graphs. To this end, we say a node v of degree three is *redundant* if the three neighbors of v are mutual neighbors of each other. (See node e in Figure 2(b) for an example.). In such a case, no shortest paths pass through v except for those that start or end at node v . We use R_3 to denote the set of such redundant nodes of degree three.

We say that a node v of degree four is redundant if the neighbors of v form a cycle of length four. In this case, removing node v from G does not affect the length of any shortest path passing through v . (See node i in Figure 2 for an example.) We use R_4 to denote the set of redundant nodes of degree four.

Figure 2 shows a complete example based on the above observations. Figure 2 shows the reciprocal of closeness-centrality values scaled by $|V| - 1 = 10$, for ease of exposition.

2.5 Maintenance

With the addition (deletion) of new edges, the structures mentioned above are likely to change. In such cases, instead of running procedures on the entire graph to identify the relevant nodes, we use book-keeping techniques and simple data structures to identify chain nodes etc. in the new graph quickly.

To this end, we maintain an array to denote the status of a node in the graph as chain, R_3, R_4 , so that if new edges are incident on these vertices, we can update them accordingly. In the case of chains, a new edge can either break a chain into multiple parts. Similarly, as per the definition of R_3 nodes, a new edge can create a new R_3 node or render an existing R_3 node as a non- R_3 node. The same applies for R_4 nodes as well. These changes can be inspected in constant time for every new edge inserted into the graph. Hence,

the total time needed to check for changes in chains, redundant nodes is linear in number of edges inserted.

3 FULLY DYNAMIC PARALLEL BETWEENNESS-CENTRALITY

In this section, we provide details on how the techniques from Section 2 can be incorporated into our algorithm for updating the betweenness-centrality values of nodes in a dynamic graph. We use *Brandes-BFS* to denote the extended BFS-like approach in the algorithm of Brandes [4]. We change the dependency accumulation formula in the algorithm of Brandes [4] suitably to account for nodes that are processed differently, as in Section 2. We then extend the approach of Jamour et al. [16] to update the betweenness-centrality values over a batch of edge updates.

3.1 Chain and Redundant Nodes

The algorithm of Brandes maintains data about the BFS traversal from each node s as the source node in four arrays: S_s, D_s, σ_s, P_s denoting the stack order of nodes visited during the BFS, the distance of nodes from the source node, the number of paths, and the parents of each node, from s respectively. Following this, the algorithm increments the betweenness-centrality of every node v by the dependency of s on v calculated as $\delta_s[v] += \frac{\sigma_s[v]}{\sigma_s[w]}(1 + \delta_s[w])$, for $v \in P_s(w)$.

Consider a chain of nodes $Q = a, s_1, s_2, \dots, s_k, b$ in a graph. We use $\sigma_{s_i a w}$ to denote the number of shortest paths from s_i to w containing a , and $\delta_{s_i}[v]$, $\delta_Q[v]$ to denote the dependency of the source node s_i and the chain Q on v respectively.

The shortest path from a chain node s_i to a node v located outside the chain must contain shortest paths a to v or b to v or both (if path $s_i - a - v$ equals path $s_i - b - v$). If we have the BFS data of both a and b , we can identify in constant time, how many such chain nodes will contain the path from a (or b) to v . So instead of running Brandes-BFS from every chain node s_i between a and b , and updating the corresponding δ_{s_i} values, we sum the dependencies of all the chain nodes on a particular node v and hence update $\delta_Q[v]$ only once using the following claims.

CLAIM 1. Let $Q = a, s_1, s_2, \dots, s_k, b$ be a chain with k chain nodes and w be a leaf node with parent node v in BFS tree of a . Let $\delta_{Q_a}[v]$ denote the dependency of the chain Q on v to reach nodes beyond v through end-point a . Then, $\delta_{Q_a}[v] = \frac{\sigma_v}{\sigma_w} \times f$ where $f = \sum_{i=1}^k \frac{\sigma_{s_i a v}}{\sigma_{s_i a v} + \sigma_{s_i b v}}$.

PROOF. For each chain node s_i , $\left(\frac{\sigma_{s_i a v}}{\sigma_{s_i a v} + \sigma_{s_i b v}}\right)$ gives the fraction of shortest paths from s_i to v that pass through a i.e it yields 1 if $d_{s_i a v} < d_{s_i b v}$, 0 if $d_{s_i a v} > d_{s_i b v}$, and a value between 0 and 1 when $d_{s_i a v} = d_{s_i b v}$. Suppose we have k chain nodes for which above fraction has a positive value. Then each s_i will have a dependency of $\frac{\sigma_v}{\sigma_w} \times \left(\frac{\sigma_{s_i a v}}{\sigma_{s_i a v} + \sigma_{s_i b v}}\right)$ on v because there is only one node beyond v i.e w . This is due to the chain structure and therefore, instead of running the BrandesBFS algorithm from each of

$s_1, s_2, \dots, s_k$ individually and calculating $\delta_{s_i}[v]$ k times, we add up the dependency of each of those k nodes and obtain the chain dependency on v through a i.e $\delta_{Q_a}[v]$. $\square$

Refer [4] for more details about definitions. For a node with non-leaf node as a child, we have Claim 2 that builds on Claim 1.

CLAIM 2. Let $Q = a, s_1, s_2, \dots, s_k, b$ be a chain with k chain nodes and u be the parent of node v mentioned in above claim. Let $\delta_{Q_a}[u]$ denote the dependency of the chain Q on u to reach nodes beyond u through a . Then, $\delta_{Q_a}[u] = \frac{\sigma_u}{\sigma_v} \times (f + \delta_{Q_a}[v])$ where $f = \sum_{i=1}^k \frac{\sigma_{s_i a u}}{\sigma_{s_i a u} + \sigma_{s_i b u}}$.

PROOF. Similar to 1, we can say that dependencies of $s_1, s_2 \dots s_k$ on u to reach its child node through a is given by $\frac{\sigma_u}{\sigma_v} \times f$ where $f = \sum_{i=1}^k \frac{\sigma_{s_i a u}}{\sigma_{s_i a u} + \sigma_{s_i b u}}$. And from previous claim, $\delta_{Q_a}[v]$ gives the dependency of same chain to reach nodes beyond v through a . But all the paths considered in $\delta_{Q_a}[v]$ may not contain node u . Since we need the dependency on u , we multiply the term by $\frac{\sigma_u}{\sigma_v}$. Adding both terms we get $\frac{\sigma_u}{\sigma_v}(f) + \frac{\sigma_u}{\sigma_v}(\delta_{Q_a}[v]) = \frac{\sigma_u}{\sigma_v} \times (f + \delta_{Q_a}[v])$. In a similar way, we can perform dependency accumulation for parent of u and so on. $\square$

The only other case to be handled is to compute the dependency of each s_i on another s_j for $s_i, s_j \in Q$. For this, we count the number of paths from s_i that use s_j and update the dependencies accordingly.

As described in Section 2.4, nodes in R_3 and those on redundant chains do not appear as intermediate nodes on shortest paths. Therefore, we proceed as follows. If any such node v is affected, we run a Brandes-BFS with v as the source, obtain a reduced graph by removing such nodes from the graph G , and perform the required Brandes-BFS traversals in the reduced graph (instead of G).

3.2 Batch Update

For updating the betweenness-centrality values for a single edge addition, Jamour et al. [16] propose the following approach. Let G be the current graph, $e = vw$ be a new edge added to G , $G' := G + e$. As described in Section 2.1, edge e affects a node u if $d(u, v) \neq d(u, w)$. We say that a BCC (*chain*) is affected if some node in that BCC (*resp.* chain) is affected.

Let L be the set of nodes affected due to the edge e . Let δ_s be the dependencies in G due to s , and δ'_s be the dependencies in G' due to s . If the BCCs of G' are not identical to that G , we obtain the BCCs of G' to proceed further using [6] for GPU and [31] for CPU. Let the edge e belong to the BCC H and let $H'_e := H + e$. Finally, let $bc_G[\]$ and $bc_{G'}[\]$ denote the betweenness-centrality values of nodes in the graph G and G' respectively. It is clear that $bc_{G'}[v] = bc_G[v] - \sum_{s \in L, s \neq v} \delta_s[v] + \sum_{s \in L, s \neq v} \delta'_s[v]$.

Jamour et al. obtain $bc_{G'}[v]$ from $bc_G[v]$ by subtracting and adding certain quantities as follows.

$$bc_{G'}[v] = bc_G[v] - A[v] + A'[v] - B[v] + B'[v] - C[v] + C'[v]$$

$$\begin{aligned}
& \text{where } A[v] = \sum_{\substack{s,t \in H'_e \\ s \neq t \neq v}} \delta_{st}[v], \quad A'[v] = \sum_{\substack{s,t \in H'_e \\ s \neq t \neq v}} \delta'_{st}[v] \\
& B[v] = \sum_{\substack{s \in G_i, t \in H'_e \\ s \neq t \neq v \\ i=1 \dots k}} \delta_{st}[v], \quad B'[v] = \sum_{\substack{s \in G_i, t \in H'_e \\ s \neq t \neq v \\ i=1 \dots k}} \delta'_{st}[v], \text{ and} \\
& C[v] = \sum_{\substack{s \in G_i, t \in G_j \\ s \neq t \neq v, i=1 \dots k \\ j=1 \dots k, i \neq j}} \delta_{st}[v], \quad C'[v] = \sum_{\substack{s \in G_i, t \in G_j \\ s \neq t \neq v, i=1 \dots k \\ j=1 \dots k, i \neq j}} \delta'_{st}[v]
\end{aligned}$$

Here G_i denotes subgraph of G connected to H'_e via articulation point i . The above approach accounts for all the source, destination pairs whose shortest paths may be affected by the new edge e both originating/terminating inside and outside H'_e . We refer the reader to [16] for a more detailed explanation of the above steps.

However, the above formula works for a single update. For a batch update that we are interested in, we now make the following observations that extend the above approach. Suppose from a batch B of edges added to G , edges in $B_S := \{e_1, e_2, \dots, e_k\}$ belong to a BCC S of G' . Let node u be affected by a subset $B_u := \{e_{i_1}, e_{i_2}, \dots, e_{i_r}\}$ of B_S .

We first remove the (old) dependencies of u in G . The new dependencies with respect to u now correspond to the dependencies in the graph $G \cup B_u$. Unlike the case of handling a single edge update, where these dependencies are computed successively in the graphs $G + e_{i_1}, G + e_{i_2}, \dots, G + e_{i_r}$, (see also Figure 1), we can update the betweenness-centrality values in the graph $G \cup B_u$ directly. In essence, we can perform one Brandes-BFS traversal from each affected node u in the graph G' .

Another optimization allows us to note that since u is not affected by the edges in $B \setminus B_u$, we can run Brandes-BFS from u in the graph G' . In light of this, for each u that is affected, we can simply run one Brandes-BFS from u in the graph G' and update the dependencies corresponding to u . As several affected nodes may be performing their Brandes-BFS traversal simultaneously and update their dependencies, we perform these updates in an *atomic* manner.

Algorithm BatchBCUpdate (**Algorithm 1**) summarizes the above discussion. We run a BFS in G from distinct endpoints of edges in B and find the affected nodes. We then subtract the dependencies of affected nodes in G . Let S and S' refer to the BCCs in G and G' that are affected by the batch update B . We process each component S individually. In the process of subtracting dependencies, we first deal with redundant chain and R_3 nodes that are affected so that they can be ignored later. Then we process chains in each component using the above claims to take advantage of the structure. From all the remaining affected nodes, we run Brandes-BFS in S and subtract the dependencies with respect to each source s . For this, we use Algorithm AtomicCentral (**Algorithm 2**) which takes inputs source s and factor $(-1$ for subtraction and $+1$ for addition) to updates $bc_{G'}[]$. **Algorithm 2** is very similar to Brandes-BFS except that instead of entire graph, it is run on a biconnected component. Hence apart from intra-component paths, we also need to account for inter-component paths which is done with help of lines 4-5, 8-9, 12-14. We add newer dependencies in G' in the same way we subtracted the old ones.

Algorithm 1: BatchBCUpdate (G, bc_G, B)

```

1  $L = \text{findAffectedNodes}(G, B)$ 
2  $bc_{G'}[v] = bc_G[v]$  for all  $v \in V$ 
3  $G' = G \cup B$ 
4 for each affected BCC  $S \in G$  do in parallel
5   Subtract dependencies  $\delta_r[ ]$  for all  $r$  such that  $r \in S$  and
    $r \in R_3 \cap L$ 
6   Add dependencies  $\delta_r[ ]$  for all  $r$  such that  $r \in S'$  and
    $r \in R_3 \cap L$ 
7  $G' = G' \setminus (R_3 \cap L)$  //  $G'$  is now the reduced graph
8 for each affected BCC  $S$  in  $G$  do
9   for each affected chain  $Q$  in  $S$  do in parallel
10    Subtract dependencies  $\delta_c[ ]$  for  $c \in Q$ 
11   for each node  $s \in S$  do in parallel
12    AtomicCentral( $s, -1$ )
13 for each affected BCC  $S'$  in  $G'$  do
14   for each affected chain  $Q$  in  $S'$  do in parallel
15    Add dependencies  $\delta_c[ ]$  for  $c \in Q$ 
16   for each node  $s \in S'$  do in parallel
17    AtomicCentral( $s, +1$ )

```

3.2.1 Running Time Analysis. When a batch B of edges is added to the graph G , the algorithm of Jamour [16] has an asymptotic time complexity in $O((\sum_{e \in B} |L(e)|) \cdot (|V| + |E|))$ where $L(e)$ is the set of nodes affected due to edge e as each update is processed individually. Using **Algorithm 1** to process the entire batch of edges at once, the asymptotic time complexity will be in $O(|L(B)| \cdot (|V| + |E|))$ where $L(B)$ denotes the set of nodes affected due to all the edges in batch B .

Delete. We handle the case of deleting a batch of edges from the current graph G similarly. In this case too, updates to the array bc are done in an atomic manner as all threads share this array.

3.3 Implementation Details

The bulk of the computation involves performing Brandes-BFS traversals from each of the affected nodes as the source. Therefore, the performance of our algorithm depends on how best we can perform multiple BFS traversals on a given graph.

On the multi-core CPU, we run one Brandes-BFS from each thread. Once Brandes-BFS from a node s finishes, we then use the arrays S_s, D_s, σ_s, P_s and update the dependencies corresponding to s . Other BFS traversals can then reuse the space for these arrays.

On the GPU, we use ideas from the BFS approach of Merrill et al. [21]. Since we need to perform multiple traversals, we run one Brandes-BFS per warp (a group of 32 threads) as this leads to improved thread utilization, space requirement, and work balance across warps. For each traversal, a warp of threads uses a shared queue to enqueue the visited nodes. This warp of threads expands the neighbor list of the node at the front of the queue to identify nodes to add to the queue. To add the discovered nodes to

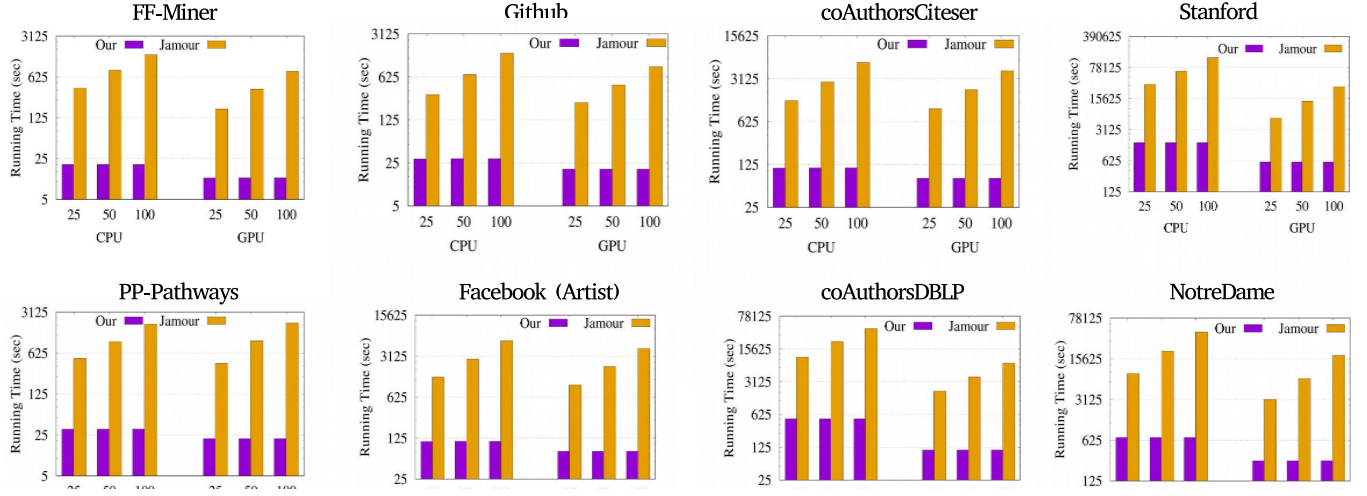


Figure 3: Plot comparing the run time of Algorithm 1 to that of Jamour et al. [16] for a batch update over batches of size 25, 50, and 100.

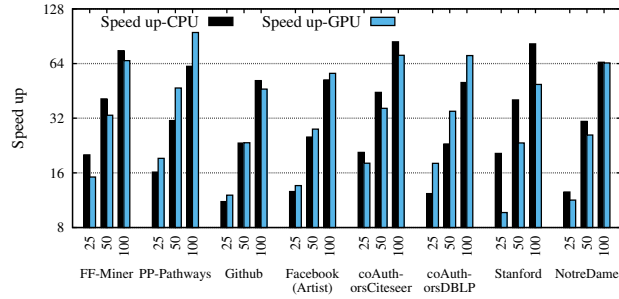


Figure 4: Plot comparing the speedup achieved by Algorithm 1 over that of Jamour et al. [16] for a batch update over batches of size 25, 50, and 100 edges on a CPU and a GPU.

Algorithm 2: AtomicCentral (s , factor)

```

1 find  $\sigma_s[v]$  and  $P_s[v]$  for  $v \in S$ 
2 for each  $v \in S$ , set  $\delta_s[v] = 0$  and  $\delta_{G_s}[v] = 0$ 
3 for  $w \in \text{reverse BFS of } s$  do
4   if  $s, w$  are articulation points then
5      $\delta_{G_s}[w] = |G_s| \cdot |G_w|$ 
6   for  $p \in P_s[w]$  do
7      $\delta_s[p] = \delta_s[p] + \frac{\sigma_s[p]}{\sigma_s[w]} \cdot (1 + \delta_s[w])$ 
8     if  $s$  is an articulation point then
9        $\delta_{G_s}[p] = \delta_{G_s}[p] + \delta_{G_s}[w] \cdot \frac{\sigma_s[p]}{\sigma_s[w]}$ 
10  if  $w \neq s$  do atomic
11     $bc_{G'}[w] += \text{factor} \cdot \delta_s[w]/2.0$ 
12  if  $s$  is an articulation point do atomic
13     $bc_{G'}[w] += \text{factor} \cdot \delta_s[w] \cdot |G_s|$ 
14     $bc_{G'}[w] += \text{factor} \cdot \delta_{G_s}[w]/2.0$ 

```

the shared queue, we use atomic operations. As the contention for atomic enqueue operation is usually small, we note that using atomic enqueue operations is efficient compared to other popular approaches, such as using prefix operations to insert the nodes into the queue at appropriate positions. Our GPU kernel groups one warp as a block of threads. On the graphs we tested on (listed in Table 3) we notice that the scheduler on the V100 GPU schedules between 250 to 300 Brandes-BFSs simultaneously across the 80 SMXs.

We handle atomic operations to the global BC array by using the openMP atomic intrinsics in the case of a multi-core CPU. On a GPU, we make use of device-level atomic operations such as *atomicAdd()*. We notice that the atomic operations on the global BC array by multiple Brandes-BFS routines usually do not update the same element of the array at the same time, so the overhead due to atomic operations is likely to be minimal. The other mechanism of using local updates followed by a scan (prefix sum) would result in more space and time overheads.

3.4 Experimental Results

3.4.1 Platform. For our experiments on a CPU, we use an Intel dual-Xeon(R) Silver 4116 processor with 128 GB RAM. This processor is a 64-bit dual-socket x86_64 architecture with 12 cores, each clocked at 2.10 GHz with hyper-threading, thereby running 48 threads at once. The processor has a three-level cache hierarchy with an L1 cache of 32 KB per core, L2 cache of 1024 KB per core and a shared L3 cache of 16.5 MB shared across 12 cores. We implement all our algorithms in C++ along with openMP version 3.1 and use the g++ version 5.4.0 compiler with the -O3 optimization flag.

We use an Nvidia Tesla V100 GPU with 16 GB of RAM for our experiments on GPUs. The V100 has 5120 cores organized as 80 Streaming Multi-processors (SMX). It has a peak single precision floating

Graph name	V	E	R_3	R_4	Chain
Biological Networks					
FF-Miner	46K	106K	0.0	0.0	9.3
PP-Pathways	35K	342K	0.0	0.0	16.2
Social Networks					
GitHub	37K	289K	4.8	2.2	12.5
Facebook(Artist)	50K	819K	3.9	4.5	6.3
Citation Networks					
coAuthorsCiteseer	227K	814K	80.4	71.9	23.6
coAuthorsDBLP	299K	977K	80.2	68.4	24.5
Web Graphs					
Stanford	281K	2009K	11.4	39.3	8.9
NotreDame	325K	1082K	11.9	16.5	10.9

Table 3: Statistics of graphs we used in our experiments. R_3 , R_4 are the percentage of nodes of degree 3 and 4 respectively. The column “Chain” corresponds to the percentage of chain nodes across the entire graph. The number of nodes and edges in the graph is approximate to the nearest thousand (K).

point performance of 15.7 TFlops and a peak memory bandwidth of 900 GBps. Each SMX also has a combined data cache and shared memory of 128 KB that is configurable as per the requirements of the workload. We use CUDA Version 9.0 for programming the V100 GPU².

3.4.2 Dataset. We experiment on four different classes of real-world graphs listed in Table 3. All the graphs are from the SNAP database³. We make each graph simple, undirected, unweighted, and connected by removing self-loops, multiple edges. We convert directed edges to undirected edges, and if the graph is disconnected, we make it connected by adding few edges.

3.4.3 Results.

Comparison to a single edge update. We first study the time taken to process a batch of updates. For each graph in Table 3, we prepare a batch of edges of a given size for insertion or deletion by choosing edges uniformly at random. If the BCCs of the current graph change, as in [16, 27], we recompute the BCCs[6, 31] of the new graph and proceed with the rest of Algorithm 1.

We compare the performance of Algorithm 1, labeled **Our** in the plots, to that of the algorithm of Jamour [16], labeled **Jamour**, in the plots. We experiment with three different batch sizes: 25, 50, and 100. The results of the experiment on the CPU and the GPU are explained in Figure 3 and 4 showing run time comparison and speedup respectively. For the latter, we implement the algorithm of Jamour [16] on a GPU. We make the following observations from Figure 3. We notice that a small set of edges usually affects a large number of node. Therefore, the number of affected nodes increases

² See <https://www.nvidia.com/en-us/data-center/tesla-v100/> for more details of the Tesla V100 GPU.

³Stanford Large Network Dataset Collection. <https://snap.stanford.edu/>.

slightly with increasing batch size. For this reason, the absolute time taken by Algorithm 1 does not increase significantly with increase in batch size.

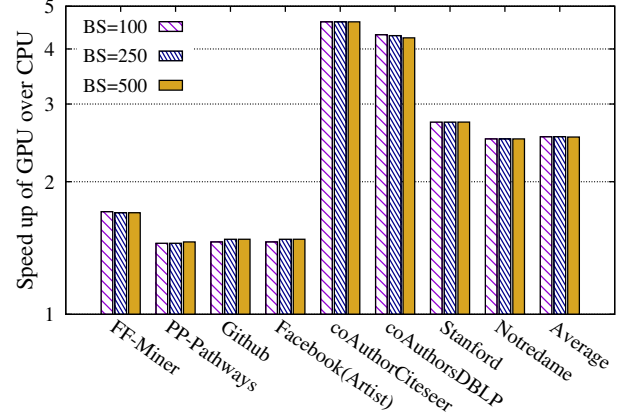


Figure 5: Relative performance of Algorithm 1 on the CPU vs. the GPU.

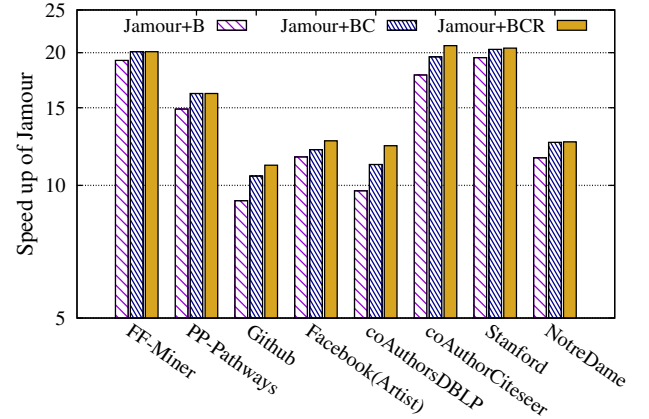


Figure 6: Plot showing the speedup, over Jamour et al. [16], of variants of algorithms. The last of these is same as Algorithm 1.

Our algorithm avoids repeated Brandes-BFS traversals from a node that is affected by more than one edge of B . Hence, we expect a good speedup compared to Jamour. Figure 4 shows the speedup of **Our** over **Jamour** for the graphs listed in Table 3 over batches of size 25, 50, and 100 edges. As can be noticed from Figure 4, for batches of size 25, 50, and 100 edges, our algorithm achieves an average speedup of 16, 32, 63, over a multi-core CPU, and a speedup of 15, 31, and 54, respectively, on a GPU. Each affected node makes atomic updates to the global BC array that stores the betweenness-centrality values. For this reason, there will be some loss in speedup.

CPU Vs. GPU. In Figure 5, we show the speedup of our GPU implementation of Algorithm 1 compared to the corresponding CPU implementation. We observe that the GPU implementation takes 2.5x lesser time compared to the CPU implementation. As the bulk of the work involved in Algorithm 1 corresponds to performing Brandes-BFS traversals from each of the affected nodes, the speedup achieved does not vary significantly with the batch size.

Variants of Algorithm 1. In Figure 6, we study how each of our techniques contribute to the overall speedup of **our** over **Jamour** on the CPU. We fix the batch size to 25 edges. Figure 6 shows that directly extending the algorithm of Jamour [16] to process a batch update, labeled **Jamour+B** in Figure 6, has an average speedup of 14x over **Jamour**. Algorithms that include additional techniques such as ignoring chain nodes (labeled **Jamour+BC**), and ignoring chain nodes and redundant nodes (labeled **Jamour+BCR**), result in an additional speedup of 1.1x each on average for the dataset in Table 3. Note that **Jamour+BCR** is equivalent to Algorithm 1 (**Our**). We observe similar results on the GPU.

4 FULLY DYNAMIC PARALLEL CLOSENESS-CENTRALITY

The closeness-centrality of a node v in a graph $G = (V, E)$, denoted $cc(v)$ is defined as $cc(v) = \frac{|V|-1}{\sum_{w \in V(G)} d(v, w)}$ where $d(v, w)$ denotes the distance between v and w in G . In this section, we show how to update the closeness-centrality values of nodes in a dynamic graph by using the techniques developed in Section 2 and extending the approach of Sarıyüce et al. [27].

4.1 Chain and Redundant Nodes

As mentioned in Section 2, for nodes that lie on chains, their closeness-centrality value can be obtained using the closeness-centrality values of the end-points of the chain. This observation allows us to ignore such nodes for further processing, even when they are affected due to an update. In addition, redundant nodes do not contribute to any of the shortest paths unless there are the source or destination of the path. Hence, if a node v is a redundant node or lies on a redundant chain, we run a BFS from such a node v and then ignore v in the subsequent BFS traversals. Doing so reduces the time spent in BFS traversals from other nodes.

4.2 Batch Update

We make use of the notion of the block-cutpoint tree (BCT) of a graph G [14]. The BCT has a node for every articulation point and BCC of G . Two nodes in the BCT are neighbors if they share an articulation point. For a node v_{bct} in the BCT T corresponding to an articulation point in G , we use $T_{v_{\text{bct}}}$ to denote the subtree of T rooted at v_{bct} .

Consider adding a batch B of edges to G and that $G' = G \cup B$. (If the BCCs of G' are not identical to that G , we recompute the BCCs of G' to proceed further.) If for a node w , $|d(u, w) - d(v, w)| \leq 1$, then

Algorithm 3: BatchCCUpdate

```

1  $L = \text{findAffectedNodes}(G, B)$ 
2  $G' = G \cup B$ ,  $T = \text{BCT}(G')$ 
3  $cc'(v) = cc(v)$  for each  $v \in G'$ 
4  $R = R_3 \cup R_4 \cup Q$  in  $G'$ 
   //  $R_3$  and  $R_4$  are redundant nodes,  $Q$  is the set
   // of chain nodes
5 for each articulation point  $a \in L$  do in parallel
6    $\lfloor$  Compute  $D'_V[a]$  through  $\text{BFS}(a, H_a)$ 
7 for each affected BCC  $S \in G'$  do in parallel
8    $\lfloor$  Compute  $D'_V[v]$  through  $\text{BFS}(v, S)$  for all  $v$  such that
   //  $v \in S$  and  $v \in L \cap R$ 
9    $\lfloor$  Compute  $D'_V[v]$  through  $\text{BFS}(v, S \setminus R)$  for all  $v$  such
   // that  $v \in S \setminus R$  and  $v \in L$ 
10  $\text{bottomUpTraversal}(T)$  // Similar to Algorithm 4
11  $\text{topDownTraversal}(T)$  // Refer Algorithm 4
12 for each BCC  $h$  in  $T$  do
13    $\lfloor$   $\Delta[ht] = \sum_i \Delta_h[a_i]$  for all articulation points  $a_i \in h$ 
14  $cc[v] = \frac{|V|-1}{D'_V[v] - \Delta[H_v]}$  for all  $v \in G$ 

```

we observe that $d(w, x)$ remains unchanged for any other node x . Therefore, we say that in the context of closeness-centrality, a node w in G is affected due to an edge $uv \in B$ iff $|d(u, w) - d(v, w)| > 1$. See also [27, Theorem 1]. We say that a biconnected component H of G' is affected if it has at least one affected node. For a BCC H_v that contains v , let L be the set of nodes in H_v and are affected due to B .

For $F \subseteq V$, let $D_F(v) := \sum_{f \in F} d(v, f)$. Then, we can rewrite $cc[v]$ as $cc[v] = \frac{|V|-1}{D_V(v)}$. Let $D'_F(v)$, $cc'(v)$ denote the corresponding quantities in the new graph G' . Then, $cc'(v) = \frac{|V|-1}{D'_V(v)}$. Further, $D'_V(v) = D_V(v) - D_L(v) + D'_L(v) + \Delta[H_v]$. The quantity $D'_L(v)$ is obtained via a BFS traversal in G' from each node in L . The quantity $\Delta[H_v]$ corresponds to the change in $D'_V(v)$ due to affected nodes in other BCCs of G and we compute $\Delta[H_v]$ as follows.

For an affected BCC H_v , let $\Delta[H_v]$ be the summation of $\Delta_{H_v}[a_i]$ for all articulation points $a_i \in H_v$ where $\Delta_{H_v}[a_i]$ is calculated as

$$\sum_{H'} \left(|V(H')| \cdot (d'(a_i, a) - d(a_i, a)) + (D'_{V(H_v)}[a] - D_{V(H_v)}[a]) \right). \quad (1)$$

In the Equation 1, H' refers to the BCCs that lie in T_{a_i} where a is the articulation point that lies in H' and in the $H_v - H'$ path in the BCT, and $V(H_v)$ refers to the vertices of G in the BCC H_v .

Consider that an articulation point a in G is also an affected node. Then $cc(w)$ may change for nodes w that are not in the BCC(s) of a . When we consider a single edge update, it can be noticed that all nodes $w \in G$ in all BCCs corresponding to nodes in the subtree T_a of the BCT of G also need to update their $cc(w)$ value. In the case of a batch update, not only nodes in BCCs in the subtree T_a of BCT

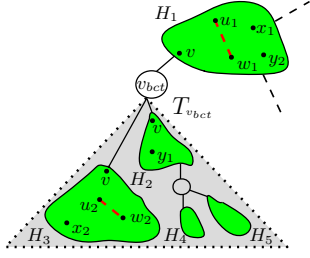


Figure 7: An illustration of how adding a batch can affect the cc values of nodes in BCC's other than the ones to which edges in B belong to. As v is affected due to new edges u_1w_1 and u_2w_2 , $d(x_1, y_1)$ and $d(x_2, y_2)$ may change requiring top-down and bottom-up traversal of the BCT respectively.

Algorithm 4: Top Down Traversal

```

1 Function topDownTraversal( $T$ ):
2    $Queue \leftarrow$  empty queue
3    $rb = \text{root}(T)$ 
4    $Queue.enqueue(A(rb))$  //  $A(H)$  is the set of
   articulation points in the BCC  $H$ 
5   while  $Queue$  is not empty do
6      $a = Queue.dequeue()$ 
7      $pb = \text{parent}[a]$ ,  $gp = \text{parent}[pb]$ 
8     for each node  $v \in A(pb) - \{gp\}$  do
9        $wt = (|V(G)| - |V(T_{gp})|)$ 
10       $\Delta_{pb}[v] = \sum_i \Delta_{pb}[a_i]$ , where  $a_i \in A(pb) - \{gp, v\}$ 
11       $\Delta_{pb}[v] = \Delta_{pb}[v] + D'_{pb}[v] - D_{pb}[v]$ 
12       $\Delta_{pb}[v] = \Delta_{pb}[v] + wt \cdot (d'(v, gp) - d(v, gp))$ 
13       $Queue.enqueue(A(\text{child}(v)) - \{v\})$ 

```

but also other nodes get impacted by node a being affected. We compute the cumulative impact on $cc(w)$ due to multiple affected nodes that are also articulation points as the sum of the individual quantities. We orchestrate this update via a bottom-up and top-down traversal of the BCT of G .

As shown in Figure 7, to populate the contributions, we start with a bottom-up traversal of the BCT. At every node v_{bct} , we accumulate the change in $D_V()$ across all the children of v_{bct} . We then perform a top-down traversal of the BCT that starts at the root and pushes the change to $D_V()$ from a parent to each of its children in the BCT. We apply the actions to nodes in the BCT to all nodes in the corresponding BCC of G in parallel. Through these two traversals, we avoid performing BFS traversals from nodes that are in BCCs that do not contain edges from B .

To summarize the above discussion, each incremental batch update consists of the steps shown in Algorithm BatchCCUpdate (**Algorithm 3**). We first find the affected nodes due to the batch B to G . We then decompose G' into its BCCs and identify nodes that can be ignored from further processing. From each affected node that is also an articulation point a in G' , we run BFS in H_a . From each

affected node v that remains, we run a BFS in H_v . Subsequently, we compute the change in distances to all nodes that lie in BCCs other than H_v by using the bottom-up and top-down traversals of the BCT. Algorithm 4 shows the pseudocode for the top-down traversal. The pseudocode for the bottom-up traversal is similar to Algorithm 4.

Running Time Analysis. The asymptotic time complexity of Algorithm 3 can be expressed similar to that of Algorithm 1.

Delete. We handle the case of deleting a batch of edges from the current graph G similarly by modifying Algorithm 3 suitably.

4.3 Implementation Details

The bulk of the computation in Algorithm 3 also involves multiple BFS traversals similar to that of Algorithm 1. Therefore, we refer the reader to Section 3.3 for implementation details concerning Algorithm 3.

4.4 Experimental Results

The experimental platform, dataset, and the setup are as mentioned in Section 3.4. Therefore, we move to describe our experiments and the results.

Comparison with a single update. We compare our work with a parallel adaptation of the work of Sariyüce et al. [27], labeled **ParSariyüce**, where each BFS is run in parallel as described in Section 3.3. For each graph in Table 3, we prepare a batch of edges of a given size for insertion or deletion by choosing edges uniformly at random. If the BCCs of the current graph change, as in [16, 27], we recompute the BCCs and proceed with the rest of Algorithm 3.

We study the run time (Figure 8) and speedup (Figure 9) achieved by Algorithm 3, labeled **Our**, over the algorithm of Sariyüce et al. [27], labeled **ParSariyüce**, across various batch sizes. For batch sizes of 100, 250, and 500 edges, our algorithm achieves an average speedup of 36, 52, 68 respectively, over a multi-core CPU, and a speedup of 48, 101, and 149 respectively, on a GPU. Our algorithm avoids repeated BFS traversals from a node that is affected by more than one edge of B resulting in a good speed up over **ParSariyüce**.

CPU Vs. GPU. We also study the speedup of our GPU based implementation of Algorithm 3 over a corresponding CPU based implementation. As Figure 10 shows, the GPU based implementation takes 5x lesser time on average over the CPU across various batch sizes.

Variations to Algorithm 3. For a batch size of 250 edges on the CPU, in Figure 11, we show how each of our techniques contributes to the overall speedup. We note from Figure 11 that extending **ParSariyüce** to process a batch update, labeled **ParSariyüce+B** in Figure 11, has an average speedup of 48x over **ParSariyüce**. Algorithms that include additional techniques such as ignoring chain nodes, labeled **ParSariyüce+BC** in Figure 11, and redundant

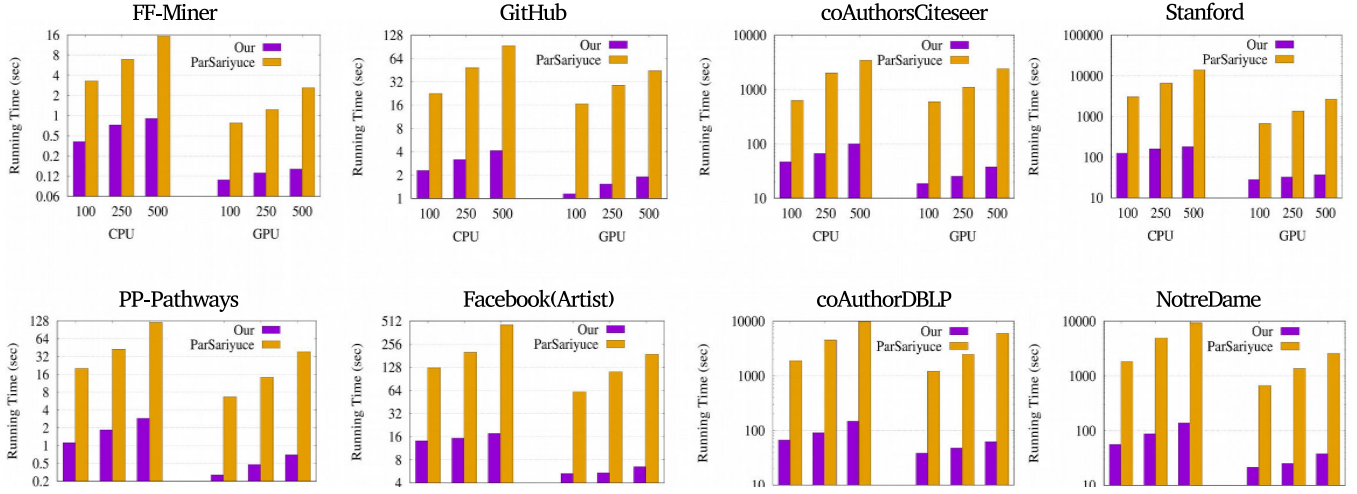


Figure 8: Plot comparing the run time of Algorithm 3 to that of Sariyüce et al. [27] for a batch update over batches of size 100, 250, and 500.

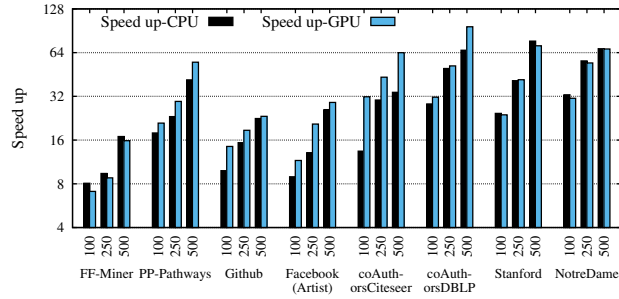


Figure 9: Plot comparing the speedup achieved by Algorithm 3 over that of Sariyüce et al. [27] for a batch update over batches of size 100, 250, and 500 edges on a CPU and a GPU.

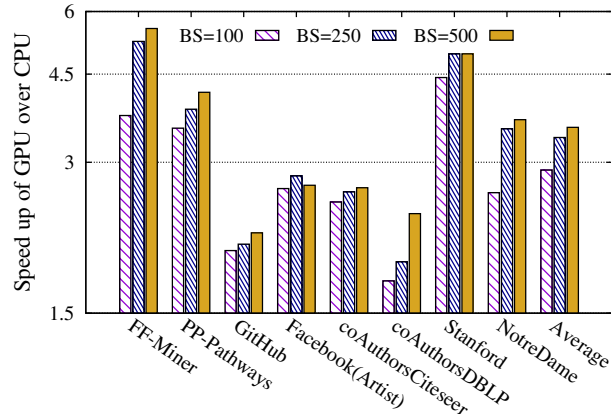


Figure 10: Relative performance of Algorithm 3 on the CPU vs. the GPU.

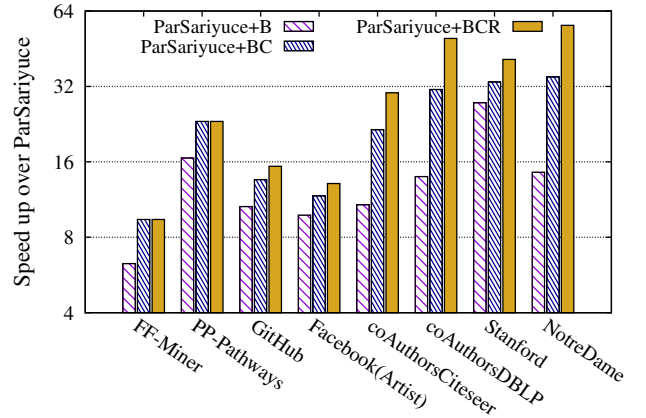


Figure 11: Plot showing the speedup, over Sariyüce et al. [27], of variants of algorithms. The last of these is same as Algorithm 3.

nodes, labeled **ParSariyuce+BCR** in Figure 11, result in an additional speedup of 1.4x and 1.2x each respectively on average for the dataset in Table 3. Note that the variant **ParSariyuce+BCR** is equivalent to Algorithm 3. We observe similar results on the GPU.

5 CONCLUSIONS

In this paper, we studied how the betweenness- and closeness-centrality values of nodes in a graph can be updated when a batch of edges are added/deleted to the current graph G . Along the way, we introduced useful preprocessing techniques, and also extend existing results to handle the case of a batch update in parallel. Our experiments indicate the efficiency of our techniques. We also notice that as the speed up of our algorithms depend largely on the

ratio of the number of traversals performed by the single update algorithm to the batch update algorithm. Therefore, the ratio of work done, and hence the speedup, depends on the input instance and is largely independent of the architecture.

In future, we would like to extend our techniques to weighted graphs and directed graphs, and also explore problems that can use the techniques introduced in this paper.

REFERENCES

- [1] David A. Bader, Shiva Kintali, Kamesh Madduri, and Milena Mihail. 2007. Approximating Betweenness Centrality. In *Algorithms and Models for the Web-Graph, 5th International Workshop, WAW 2007, San Diego, CA, USA, December 11–12, 2007, Proceedings (Lecture Notes in Computer Science)*, Anthony Bonato and Fan R. K. Chung (Eds.), Vol. 4863. Springer, 124–137. https://doi.org/10.1007/978-3-540-77004-6_10
- [2] Alex Bavelas. 1948. A Mathematical Model for Group Structures. In *Human Organization: Summer 1948, Vol. 7, No. 3*. SFAA, 16–30. <https://doi.org/10.17730/humo.7.3.f4033344851gl053>
- [3] Elisabetta Bergamini, Henning Meyerhenke, and Christian Staudt. 2015. Approximating Betweenness Centrality in Large Evolving Networks. In *Proceedings of the Seventeenth Workshop on Algorithm Engineering and Experiments, ALENEX 2015, San Diego, CA, USA, January 5, 2015*, Ulrik Brandes and David Eppstein (Eds.). SIAM, 133–146. <https://doi.org/10.1137/1.9781611973754.12>
- [4] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. In *Journal of Mathematical Sociology, Vol. 25, Issue 2*. 163–177. <https://doi.org/10.1080/0022250X.2001.9990249>
- [5] Federico Busato, Oded Green, Nicola Bombieri, and David A. Bader. 2018. Hornet: An Efficient Data Structure for Dynamic Sparse Graphs and Matrices on GPUs. In *2018 IEEE High Performance Extreme Computing Conference, HPEC 2018, Waltham, MA, USA, September 25–27, 2018*. IEEE, 1–7. <https://doi.org/10.1109/HPEC.2018.8547541>
- [6] Meher Chaitanya and Kishore Kothapalli. 2016. Efficient Multicore Algorithms For Identifying Biconnected Components. *Int. J. Netw. Comput.* 6, 1 (2016), 87–106. <http://www.ijnc.org/index.php/ijnc/article/view/119>
- [7] Edith Cohen, Daniel Delling, Thomas Pajor, and Renato F. Werneck. 2014. Computing classic closeness centrality, at scale. In *Proceedings of the second ACM conference on Online social networks, COSN 2014, Dublin, Ireland, October 1–2, 2014*, Alessandra Sala, Ashish Goel, and Krishna P. Gummadi (Eds.). ACM, 37–50. <https://doi.org/10.1145/2660460.2660465>
- [8] Hristo Djidjev, Guillaume Chapuis, Rumen Andonov, Sunil Thulasidasan, and Dominique Lavenier. 2015. All-Pairs Shortest Path algorithms for planar graph for GPU-accelerated clusters. *J. Parallel Distributed Comput.* 85 (2015), 91–103. <https://doi.org/10.1016/j.jpdc.2015.06.008>
- [9] David Ediger, Robert McColl, E. Jason Riedy, and David A. Bader. 2012. STINGER: High performance data structure for streaming graphs. In *IEEE Conference on High Performance Extreme Computing, HPEC 2012, Waltham, MA, USA, September 10–12, 2012*. IEEE, 1–5. <https://doi.org/10.1109/HPEC.2012.6408680>
- [10] David Eppstein, Zvi Galil, Giuseppe F. Italiano, and Amnon Nissenzeig. 1997. Sparsification - a technique for speeding up dynamic graph algorithms. *J. ACM* 44, 5 (1997), 669–696. <https://doi.org/10.1145/265910.265914>
- [11] Paolo Ferragina. 1994. Static and Dynamic Parallel Computation of Connected Components. *Inf. Process. Lett.* 50, 2 (1994), 63–68. [https://doi.org/10.1016/0020-0190\(94\)00016-6](https://doi.org/10.1016/0020-0190(94)00016-6)
- [12] Greg N. Frederickson. 1985. Data Structures for On-Line Updating of Minimum Spanning Trees, with Applications. *SIAM J. Comput.* 14, 4 (1985), 781–798. <https://doi.org/10.1137/0214055>
- [13] Oded Green, Robert McColl, and David A. Bader. 2012. A Fast Algorithm for Streaming Betweenness Centrality. In *2012 International Conference on Privacy, Security, Risk and Trust, PASSAT 2012, and 2012 International Conference on Social Computing, SocialCom 2012, Amsterdam, Netherlands, September 3–5, 2012*. IEEE Computer Society, 11–20. <https://doi.org/10.1109/SocialCom-PASSAT.2012.37>
- [14] Frank Harary. 1991. *Graph theory*. Addison-Wesley.
- [15] Sungpack Hong, Nicole C. Rodia, and Kunle Olukotun. 2013. On fast parallel detection of strongly connected components (SCC) in small-world graphs. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC'13, Denver, CO, USA - November 17 - 21, 2013*, William Gropp and Satoshi Matsuo (Eds.). ACM, 92:1–92:11. <https://doi.org/10.1145/2503210.2503246>
- [16] Fuad T. Jamour, Spiros Skiadopoulos, and Panos Kalnis. 2018. Parallel Algorithm for Incremental Betweenness Centrality on Large Graphs. *IEEE Trans. Parallel Distrib. Syst.* 29, 3 (2018), 659–672. <https://doi.org/10.1109/TPDS.2017.2763951>
- [17] Miray Kas, Kathleen M. Carley, and L. Richard Carley. 2013. Incremental closeness centrality for dynamically changing social networks. In *Advances in Social Networks Analysis and Mining 2013, ASONAM '13, Niagara, ON, Canada - August 25 - 29, 2013*, Jon G. Rokne and Christos Faloutsos (Eds.). ACM, 1250–1258. <https://doi.org/10.1145/2492517.2500270>
- [18] Gueorgi Kossinets and Duncan J. Watts. 2006. Empirical Analysis of an Evolving Social Network. *Science* 311, 5757 (2006), 88–90. <https://doi.org/10.1126/SCIENCE.1116869>
- [19] Robert McColl, Oded Green, and David A. Bader. 2013. A new parallel algorithm for connected components in dynamic graphs. In *20th Annual International Conference on High Performance Computing, HiPC 2013, Bengaluru (Bangalore), Karnataka, India, December 18–21, 2013*. IEEE Computer Society, 246–255. <https://doi.org/10.1109/HiPC.2013.6799108>
- [20] Adam McLaughlin and David A. Bader. 2014. Scalable and High Performance Betweenness Centrality on the GPU. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2014, New Orleans, LA, USA, November 16–21, 2014*, Trish Damkroger and Jack J. Dongarra (Eds.). IEEE Computer Society, 572–583. <https://doi.org/10.1109/SC.2014.52>
- [21] Duane Merrill, Michael Garland, and Andrew S. Grimshaw. 2012. Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2012, New Orleans, LA, USA, February 25–29, 2012*, J. Ramanujam and P. Sadayappan (Eds.). ACM, 117–128. <https://doi.org/10.1145/2145816.2145832>
- [22] Charudatt Pachorkar, Meher Chaitanya, Kishore Kothapalli, and Debajyoti Bera. 2016. Efficient Parallel Ear Decomposition of Graphs with Application to Betweenness-Centrality. In *23rd IEEE International Conference on High Performance Computing, HiPC 2016, Hyderabad, India, December 19–22, 2016*. IEEE Computer Society, 301–310. <https://doi.org/10.1109/HiPC.2016.043>
- [23] Matteo Pontecorvi and Vijaya Ramachandran. 2015. A Faster Algorithm for Fully Dynamic Betweenness Centrality. *CoRR* abs/1506.05783 (2015). [arXiv:1506.05783](http://arxiv.org/abs/1506.05783) <http://arxiv.org/abs/1506.05783>
- [24] G. Ramalingam. 1996. *Bounded Incremental Computation*. Lecture Notes in Computer Science, Vol. 1089. Springer. <https://doi.org/10.1007/BFb0028290>
- [25] Liam Roditty and Uri Zwick. 2016. A Fully Dynamic Reachability Algorithm for Directed Graphs with an Almost Linear Update Time. *SIAM J. Comput.* 45, 3 (2016), 712–733. <https://doi.org/10.1137/13093618X>
- [26] Ahmet Erdem Sariyüce, Kamer Kaya, Erik Saule, and Ümit V. Çatalyürek. 2017. Graph Manipulations for Fast Centrality Computation. *ACM Trans. Knowl. Discov. Data* 11, 3 (2017), 26:1–26:25. <https://doi.org/10.1145/3022668>
- [27] Ahmet Erdem Sariyüce, Erik Saule, Kamer Kaya, and Ümit V. Çatalyürek. 2013. STREAMER: A distributed framework for incremental closeness centrality computation. In *2013 IEEE International Conference on Cluster Computing, CLUSTER 2013, Indianapolis, IN, USA, September 23–27, 2013*. IEEE Computer Society, 1–8. <https://doi.org/10.1109/CLUSTER.2013.6702680>
- [28] George M. Sloba and Kamesh Madduri. 2014. Simple parallel biconnectivity algorithms for multicore platforms. In *21st International Conference on High Performance Computing, HiPC 2014, Goa, India, December 17–20, 2014*. IEEE Computer Society, 1–10. <https://doi.org/10.1109/HiPC.2014.7116914>
- [29] Sriram Srinivasan, Sanjukta Bhowmick, and Sajal K. Das. 2016. Application of Graph Sparsification in Developing Parallel Algorithms for Updating Connected Components. In *2016 IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPS Workshops 2016, Chicago, IL, USA, May 23–27, 2016*. IEEE Computer Society, 885–891. <https://doi.org/10.1109/IPDPSW.2016.180>
- [30] Sriram Srinivasan, Sara Riaz, Boyana Norris, Sajal K. Das, and Sanjukta Bhowmick. 2018. A Shared-Memory Parallel Algorithm for Updating Single-Source Shortest Paths in Large Dynamic Networks. In *25th IEEE International Conference on High Performance Computing, HiPC 2018, Bengaluru, India, December 17–20, 2018*. IEEE, 245–254. <https://doi.org/10.1109/HiPC.2018.00035>
- [31] Mihir Wadvekar and Kishore Kothapalli. 2017. A fast GPU algorithm for biconnected components. In *Tenth International Conference on Contemporary Computing, IC3 2017, Noida, India, August 10–12, 2017*, Srinivas Alum, Ananth Kalyanaraman, Bora Uçar, Kishore Kothapalli, Mahantesh Halappanavar, Kamesh Madduri, Madhu Govindaraju, Yinglong Xia, Sushil K. Prasad, Martina Barnas, Ashish Sureka, Pankesh Patel, Vikas Saxena, and Sanjay Goel (Eds.). IEEE Computer Society, 1–6. <https://doi.org/10.1109/IC3.2017.8284293>
- [32] Chia-Chen Yen, Mi-Yen Yeh, and Ming-Syan Chen. 2013. An Efficient Approach to Updating Closeness Centrality and Average Path Length in Dynamic Networks. In *2013 IEEE 13th International Conference on Data Mining, Dallas, TX, USA, December 7–10, 2013*, Hui Xiong, George Karypis, Bhavani M. Thuraisingham, Diane J. Cook, and Xindong Wu (Eds.). IEEE Computer Society, 867–876. <https://doi.org/10.1109/ICDM.2013.135>